Object-Oriented Programming I

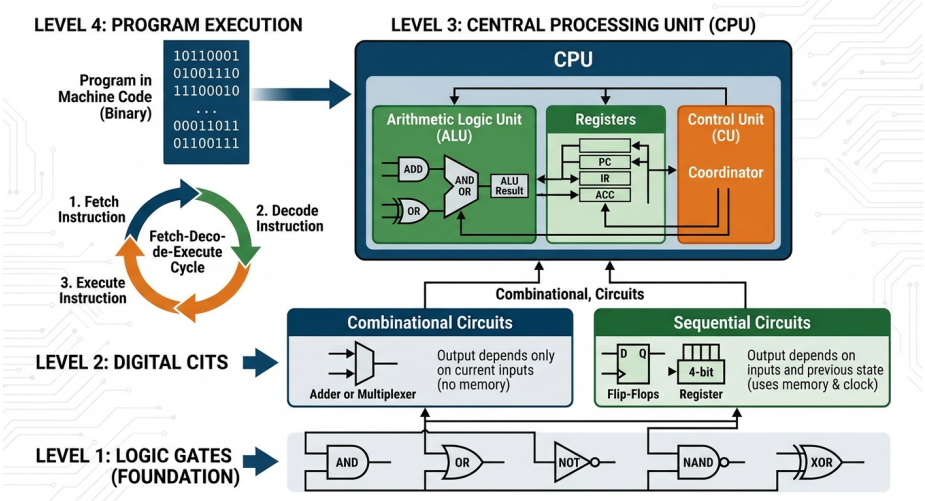
Lecture 1: Introduction

Dr. Ghemougui Abdessettar

University of M'Sila – Faculty of Mathematics and Computer Science

September 29, 2026

Program Execution



Machine Language

- Binary instructions executed **directly** by the CPU.
- Instructions load/store data, branch, or perform arithmetic/logic operations on registers and memory.
- Very close to the hardware, **architecture-specific**, and hard for humans to read or write.

```
10110000 01100100      B0 64
00000011 11000001  ~   03 C1
10100011 11001000      A3 C8
```

Assembly Language

Definition

A low-level language using human-readable **mnemonics** instead of raw binary/hex codes.

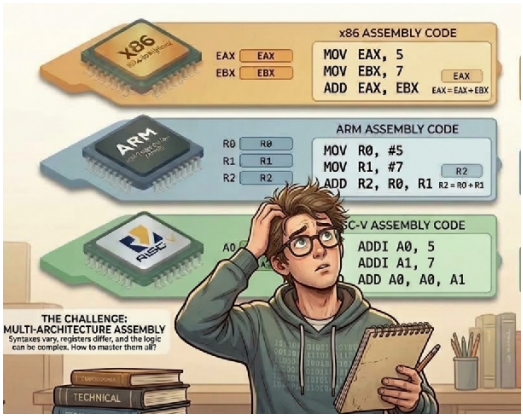
- Strong, generally **one-to-one** correspondence with machine instructions.
- Translated to machine code by an **assembler**.
- Easier to read than machine code, but still tied to one processor's instruction set and registers.

```
LOAD   R1, 100
ADD    R1, R2
STORE  R1, 200
```

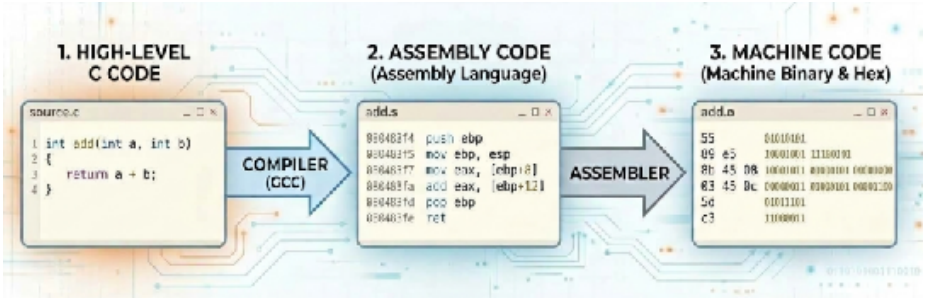
The Problem

The problem:

- **A new language per processor** — x86, ARM, and MIPS each have their own instructions, registers, and conventions.
- **No portability** — code for one architecture must be rewritten from scratch for another.
- **Does not scale** — real programs are millions of lines; rewriting all of it per target is slow and error-prone.



The Solution

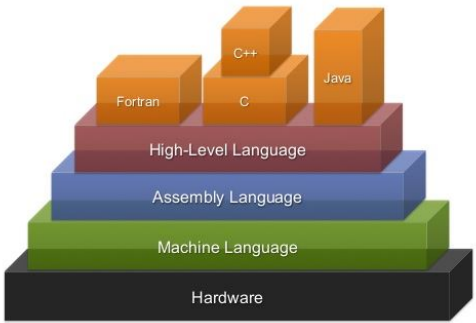


Key Idea

Write the task once in a high-level language, and let a **compiler** translate it for the target processor.

High-Level Programming Languages

- Programmer focuses on **algorithms, data, problem-solving, structure** — not registers/addresses.
- Emerged in the 1950s and 1960s: FORTRAN, ALGOL, COBOL, BASIC → later C, Pascal, ..., C++, Java, Python.
- Translated to machine code by a **compiler**, or executed by an **interpreter**.



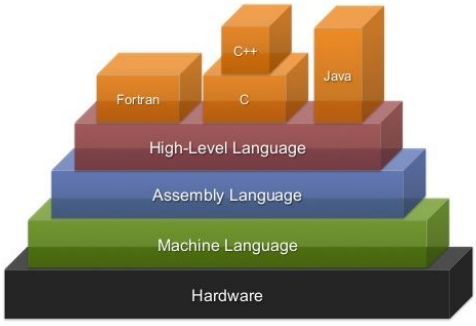
Abstraction

Definition

Abstraction is the intentional suppression or hiding of details of a process or artifact, in order to highlight other essential aspects, details, or structures more clearly.

Key Idea

The higher the level of abstraction, the further the programmer is from the physical details of the machine.



Programming Paradigms

Definition

A **programming paradigm** is a general way of thinking about and structuring programs: how to represent a problem, organize a solution, and choose the program's main elements.

Paradigm	**Main idea**
Procedural	Procedures/functions performing operations
Object-oriented	Interacting objects, state + behavior
Functional	Functions and transformations
Declarative	Describe *what*, not *how*

One Language, Several Paradigms

Language	Category	Common paradigms
C	Multi-paradigm	Procedural
C++	Multi-paradigm	Procedural, object-oriented
Java	Multi-paradigm	Object-oriented
C#	Multi-paradigm	Object-oriented, functional
Python	Multi-paradigm	Procedural, OO, functional
JavaScript	Multi-paradigm	Procedural, OO, functional

Note

A **language** is a tool for expressing programs; a **paradigm** is a way of organizing and reasoning about them. Our focus in this course is the **object-oriented paradigm**.

Procedural Programming

A program organized around procedures/functions describing the operations to perform on data.

```
1. Read the student's grades
2. Calculate the sum
3. Calculate the average
4. Determine whether the student passed
5. Display the result
```

Key Idea

Focus on *what the program has to do*.

Characteristics of Procedural Programming

1. Emphasis on algorithms and operations.
2. Decomposition into functions.
3. Data is passed between functions.
4. Data may be shared (e.g. global variables).
5. Functions transform data.
6. Top-down design.

Limitations: Global Data

```
1 double balance;
2
3 void deposit(double amount)  { balance += amount; }
4 void withdraw(double amount) { balance -= amount; }
5
```

In a program with hundreds of functions: which ones can modify `balance`? What if one changes it by accident? As the program grows, controlling data/function relationships becomes hard.

Limitations: Lack of Data Hiding

```
1 double balance;
2
```

This declaration does not express:

"Only the bank account operations should be allowed to modify this value."

Note

Not every procedural program uses global data — but the paradigm itself provides no built-in mechanism for bundling state with its permitted operations.

Limitations: Modeling Complex Entities

A university system contains real-world entities: `Student`, `Teacher`, `Course`, `Department`, ...

```
registerStudent(...);
enrollStudent(...);
withdrawStudent(...);
displayStudent(...);
updateStudent(...);

```

As the system grows, keeping data and the operations that belong to the same conceptual entity together becomes increasingly difficult in a purely procedural design.

The Code Reuse Problem

Another program needs almost the same functionality that you have already written.
Options:

1. Copy the existing code.
2. Rewrite the functionality.
3. Create a reusable library.
4. Design reusable abstractions.

A bug in copied code now has to be fixed in every copy.

Key Idea

Duplicate code is difficult to maintain.

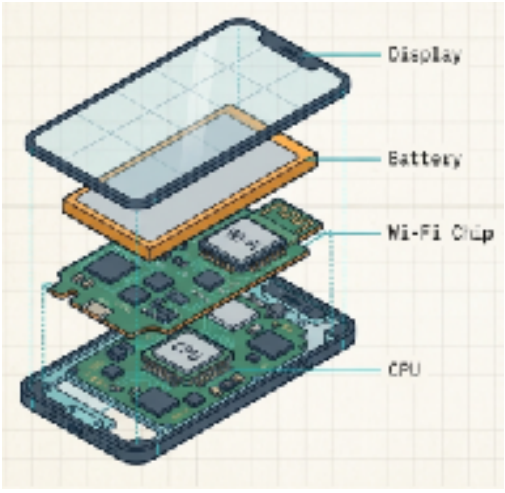
Thinking in Terms of Objects

A smartphone is not a single indivisible component:

- A smartphone's hardware is built from existing components.
- The manufacturer does not need to know how each component works or how it was made; it only needs to know how to interface with it.

Key Idea

What if we could build software in the same way?

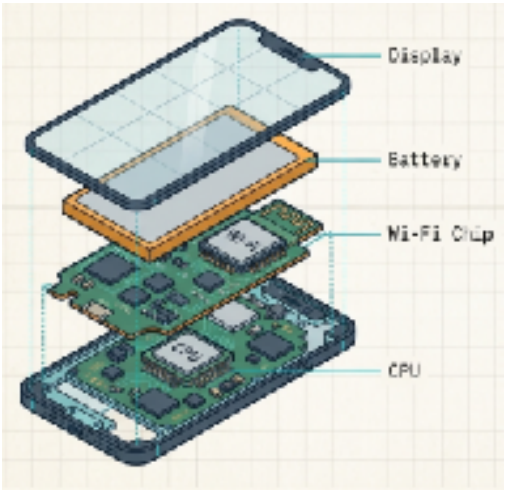


Thinking in Terms of Objects

Components are bought or reused; the manufacturer only needs to know how to *interface* with each one, not how it works inside.

Key Idea

Complex systems can be built by combining reusable components.



From Hardware Components to Software Objects

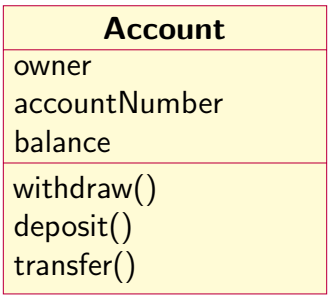
What if a program were built from independent, self-contained components — each bundling its own data and the behavior needed to act on it safely? Each exposes only an **interface** and hides its **implementation**.

Key Idea

Modeling a program as a collection of interacting objects, each responsible for its own data and behavior.

Example: An `Account` Object

Data and the operations that work on it are brought together into a single conceptual unit.



A Different Way of Thinking

Procedural

Key Idea

What operations does the program need to perform?

Object-oriented

Key Idea

What entities exist, what do they know, and what can they do?

Key Idea

A program can be viewed as a collection of objects that collaborate to solve a problem.

From Reusable Components to OOP

Questions this raises:

- How do we define a reusable object, and protect its data?
- How do we create many objects of the same kind?
- How can one object be based on another?
- How can different objects respond differently to the same operation?
- How can we reuse code without copying it?

Mechanisms we will study: **classes and objects, encapsulation, abstraction, inheritance, polymorphism**.

The Goal of This Course

Key Idea

Learn how to identify the important entities and abstractions in a problem, model their relationships and behavior, and implement that model as a maintainable software system.

Basics of Object-Oriented Programming

Key Idea

In this course, we will learn the basics of the OOP paradigm and how to design and implement software using these concepts.

- Object
- Class
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Messaging